

Group 12

Members:

Name: 謝尚恆 Student ID: M11215101

Name: Guérin Ulysse Student ID: E11315012

Name: MENDEZ SALGADO LUIS ANDRES Student ID: D11215801

Name: 吳浚瑋 student ID: M11215057

Assignment Midterm – Differential Privacy

1. Introduction

In recent years, deep learning models have achieved significant success in a wide range of applications, including image classification, natural language processing, and medical diagnosis. However, training these models often requires access to sensitive data, raising concerns about data privacy and user confidentiality.

Differential Privacy (DP) has emerged as a formal framework to provide mathematical guarantees that protect individual data points from being exposed during model training and inference. This project explores the integration of differential privacy into deep learning workflows using the **Opacus** library in PyTorch. The goal is to evaluate the trade-off between model utility and privacy when applying DP to a neural network trained on the MNIST dataset.

2. Background and Motivation

Differential Privacy (DP) is a mathematical framework designed to protect individual data while enabling useful data analysis. It ensures that the output of an algorithm remains statistically similar, regardless of whether any single individual's data is included in the input dataset. This is achieved by adding calibrated noise during the training process, typically to gradients, model parameters, or outputs. This helps mitigate the risk of re-identifying specific individuals, even when adversaries have auxiliary information.

A key measure in DP is the **privacy budget**, denoted by ϵ (**epsilon**). Smaller values of ϵ offer stronger privacy protection by minimizing the influence of individual data points on the model's output. However, stronger privacy generally leads to reduced model accuracy, as too much noise can degrade the learning

process. Balancing this trade-off is crucial, especially in privacy-sensitive domains such as healthcare, finance, and personalized services.

In this project, we use the **Opacus** library, a PyTorch-based framework for efficient differentially private deep learning, to implement **Differentially Private Stochastic Gradient Descent (DP-SGD)**. DP-SGD is widely used for training neural networks under privacy constraints. Unlike standard SGD, DP-SGD clips gradients and injects noise to obscure the presence of specific data points, ensuring privacy while enabling model training on sensitive datasets.

3. Methodology

3.1 Dataset

The dataset used for experimentation is **MNIST**, a benchmark dataset consisting of 70,000 grayscale images of handwritten digits (60,000 for training, 10,000 for testing), each sized 28×28 pixels.

3.2 Framework and Libraries

- PyTorch: For deep learning model implementation and training.
- Opacus: To integrate Differential Privacy into the PyTorch training loop
- TensorBoard: For visualization of training metrics including loss, accuracy, and ϵ .

3.3 Differential Privacy Implementation

The project uses **DP-SGD**, implemented through Opacus's PrivacyEngine. It works by:

- **Clipping** the per-sample gradients to a fixed norm (1.0 in our setup)
- **Adding Gaussian noise** to the aggregated gradients
- **Tracking ϵ (epsilon)** over training to monitor the privacy budget

The DP engine wraps the model, optimizer, and data loader to enforce privacy during training. The value of ϵ is calculated using the provided `get_epsilon()` method and logged throughout training.

3.4 SimpleNN Model Architecture

The first part of the project uses a simple fully connected neural network (SimpleNN) with one hidden layer. It includes:

- **Input layer**
An input layer that flattens 28×28 pixel images into 784 dimensional vectors.
 - **Hidden layer**
A hidden layer with 256 neurons and ReLU activation
 - **Dropout layer**
Dropout with a rate of 0.3 to prevent overfitting and improve generalization.
 - **Output layer**
An output layer with 10 neurons for classification.
The final output of the model is a vector of 10 values, each corresponding to the model's confidence that the input image belongs to a particular class.
- Input layer and Output layer are both fully connected layers.

3.5 CNN Model Architecture

In the second part of the project, a Convolutional Neural Network (CNN) is implemented to improve the classification accuracy under differential privacy constraints. The CNN is structured as follows:

- **Convolutional Layers:**
The first convolutional layer has 32 filters and the second layer 64 filters. Both layers have a kernel size of 3x3 and padding of 1.
ReLU activation functions are applied after each convolutional layer.
The stride is set to 1 in both layers, which ensures that the spatial dimensions are reduced gradually.
- **Pooling:**
A max pooling layer with a 2x2 kernel size is applied after the second convolutional layer. This layer reduces the spatial dimensions of the feature maps from 28x28 to 14x14, retaining only the most significant features and helping the model become invariant to small translations.
- **Dropout Regularization:**
Dropout is applied with a rate of 0.25 after the pooling and before the fully connected layers to prevent overfitting.
- **Fully connected Layers:**
After feature extraction, the flattened feature maps are passed through a fully connected layer with 128 hidden units.
Another dropout layer is applied before the output layer to further prevent overfitting.
The final layer contains 10 outputs neurons corresponding to the 10 digits classes.

This architecture allows the model to learn spatial features from the MNIST images more effectively while still complying with differential privacy requirements through Opacus.

4. Experimental Results

4.1 Experimental Results with Simple Neural Network (SNN)

Model Architecture:

The initial experiments were conducted using a fully connected neural network (SimpleNN) composed of the following layers:

- Input layer: 28×28 flattened MNIST images
- Hidden layer: Linear (784 → 256) + ReLU activation
- Dropout: 30% to improve generalization under privacy noise
- Output layer: Linear (256 → 10) for digit classification

Training Setup:

- Optimizer: DP-SGD (Differentially Private Stochastic Gradient Descent)
- Learning rate: 0.1 (with StepLR scheduler, gamma = 0.5 every 2 epochs)
- Loss Function:CrossEntropyLoss (CE Loss)
- Noise multiplier: 1.1
- Batch size: 64
- Epochs: 10
- Clipping norm: 1.0
- Dataset: MNIST (60,000 training, 10,000 test images)
- Privacy budget (δ): 1e-5

Performance Evaluation:

The model was evaluated using two key metrics:

- **Accuracy** on the test set (effectiveness)
- **Epsilon (ϵ)**, the differential privacy budget (privacy guarantee)

Epoch	Avg Loss	Accuracy	Epsilon
1	0.8454	86.58%	0.14

2	0.7086	88.19%	0.19
3	0.7305	88.86%	0.23
4	0.7429	89.38%	0.27
5	0.7237	89.88%	0.30
6	0.6937	90.11%	0.33
7	0.6983	90.30%	0.36
8	0.6818	90.43%	0.38
9	0.6797	90.46%	0.40
10	0.6886	90.56%	0.43

Analysis:

- The model achieved **over 90% accuracy** while maintaining a very low privacy budget ($\epsilon < 0.5$), which is considered strong privacy in most applications.
- Loss decreased and stabilized after several epochs, confirming the effectiveness of the chosen optimizer and learning rate schedule.
- These results demonstrate that differential privacy can be effectively integrated into model training without drastically sacrificing performance.

4.2 Experimental Results with Convolutional Neural Network (CNN)

In this experiment, a Convolutional Neural Network (CNN) architecture was evaluated under the same differentially private training setup as the Simple Neural Network (SNN). The architecture consists of two convolutional layers followed by max pooling, dropout layers, and two fully connected layers. This structure allows the network to capture spatial hierarchies and local patterns within the MNIST digit images, which is particularly advantageous for image classification tasks.

The CNN was trained using DP-SGD with a noise multiplier of 1.1, a clipping norm of 1.0, and a batch size of 64 for 10 epochs. The training loss used CrossEntropyLoss. The training process used the Opacus library to integrate differential privacy mechanisms, and performance was monitored through both accuracy and privacy budget (ϵ) progression.

Epoch	Avg Loss	Accuracy	Epsilon
1	1.3453	87.30%	0.14
2	1.6027	88.64%	0.19
3	1.5222	89.33%	0.23
4	1.3659	89.10%	0.27
5	1.2708	90.33%	0.30
6	1.2008	90.30%	0.33

7	1.1743	90.68%	0.36
8	1.1141	90.86%	0.38
9	1.1077	91.12%	0.40
10	1.0691	91.13%	0.43

Compared to the SNN, the CNN demonstrated slightly higher accuracy throughout the training process, reaching 91.13% by the final epoch. Although the average loss values were higher than those observed with the SNN, the classification performance improved. This highlights a key point: while the CNN may have produced less confident predictions on some samples (contributing to a higher loss), it was generally more successful at making correct predictions.

Furthermore, the CNN reached the same privacy budget of $\epsilon \approx 0.43$ as the SNN, confirming that the improved accuracy did not come at the cost of reduced privacy protection.

In summary, the CNN yielded superior classification performance under identical differential privacy constraints, indicating that convolutional architectures are more robust and effective for privacy-preserving image classification.

4.2.1 What happens if the number of epochs is doubled?

To further analyze the privacy-utility tradeoff, the CNN model was trained for 20 epochs using the same DP-SGD configuration. This resulted in a marginal improvement in test accuracy, increasing from 91.13% to 91.43%, while the privacy budget (ϵ) rose from 0.43 to 0.61. This result confirms that longer training allows the model to converge further, albeit with diminishing returns in utility and increased privacy cost. These findings underscore the importance of carefully selecting the number of training epochs to balance performance and privacy, depending on the sensitivity of the dataset and the requirements of the application.

What happens without differential privacy?

When training a simple neural network without implementing differential privacy, we get slightly better results than with differential privacy, because of the less demanding conditions, each entry can be utilized to its fullest. With the same number of 10 epochs, we obtain an accuracy of approximately 98%, which is better than previously but might not be worth the privacy loss.

This model uses SGD too to be a pertinent comparison point.

Test Accuracy: 98.24%

5. Comparison of SimpleNN and CNN Performance Under Differential Privacy

To evaluate the effectiveness of different neural network architectures in a privacy-preserving context, both a simple fully connected neural network (SimpleNN) and a convolutional neural network (CNN) were trained on the MNIST dataset using differentially private stochastic gradient descent (DP-SGD). Each model was trained for 10 epochs with the same differential privacy parameters: noise multiplier = 1.1, maximum gradient norm = 1.0, and a target delta of $1e-5$. The privacy budget (ϵ) reached approximately 0.43 for both models.

The results indicate that the CNN consistently outperformed the SimpleNN in terms of classification accuracy. By the end of training, the CNN achieved an accuracy of 91.13%, compared to 90.56% with the SimpleNN. This demonstrates the superior representational power of convolutional layers in learning spatial patterns, even under privacy constraints.

It is notable, however, that the CNN reported a higher average loss (1.0691) than the SimpleNN (0.6886). This discrepancy is likely due to the nature of the cross-entropy loss, which penalizes incorrect and low-confidence predictions more heavily. The CNN, while making more accurate predictions, may have produced outputs with less confident probability distributions, leading to a higher loss. Nonetheless, since accuracy is the primary evaluation metric in this classification task, the CNN is considered the better-performing model.

Model	Final Accuracy	Final Loss	Epsilon
SimpleNN	90.56%	0.6886	0.43
CNN	91.13%	1.0691	0.43

An important observation from the experiments is that both the Simple Neural Network (SNN) and the Convolutional Neural Network (CNN) reached identical ϵ (epsilon) values at each epoch during training. This may seem unexpected given the architectural differences between the two models. However, this is explained by the fact that the privacy budget (ϵ) in differential privacy depends solely on a set of parameters related to how the data is accessed, rather than the specific structure of the model. These parameters include the noise multiplier, clipping norm, batch size, dataset size, and the number of

training steps. Since both models were trained using the same DP-SGD configuration—with identical values for the noise multiplier, clipping norm, batch size, and total number of epochs—their ϵ values evolved identically. This highlights an important property of differential privacy: the privacy guarantee is determined by the training procedure rather than the model complexity.

In conclusion, while both models achieved high accuracy under a strict privacy budget, the CNN demonstrated a marginal improvement in accuracy, confirming its effectiveness for differentially private image classification tasks.

6. Conclusion

This study explored the application of Differentially Private Stochastic Gradient Descent (DP-SGD) in training machine learning models on the MNIST dataset. Two architectures—a Simple Neural Network (SNN) and a Convolutional Neural Network (CNN)—were implemented and evaluated using the Opacus library to enforce differential privacy guarantees.

Both models were trained under the same privacy parameters, achieving a comparable privacy budget of $\epsilon \approx 0.43$ after 10 epochs. The experimental results demonstrated that the CNN outperformed the SNN in terms of classification accuracy, reaching 91.13% compared to the SNN's 90.56%, despite the CNN reporting a higher average training loss. This suggests that CNNs are more effective at capturing spatial patterns and generalizing under privacy constraints.

Further experimentation with extended training for the CNN showed that increasing the number of epochs to 20 provided a marginal improvement in accuracy (up to 91.43%) but came at the cost of a higher privacy budget ($\epsilon \approx 0.61$). This highlights the fundamental trade-off between model utility and privacy when applying differential privacy in practice.

In conclusion, CNNs offer a performance advantage in differentially private learning settings, and the balance between training duration and privacy budget must be carefully managed to meet the specific requirements of privacy-sensitive applications.

APPENDIX

Video link: <https://www.youtube.com/watch?v=gWPZjl3JgkI>

Contributions

Ulysse Guérin E11315012	Slides, script and results for the Non-dp script.
謝尚恆 M11215101	Video recording
MENDEZ SALGADO LUIS ANDRES D11215801	Redacting report and DP python scripts
吳浚瑋 M11215057	modify document

SimpleNN Script

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torch.utils.tensorboard import SummaryWriter
from opacus import PrivacyEngine
import os

# -----
# Hyperparameters
# -----
EPOCHS = 10          # Number of training epochs
BATCH_SIZE = 64      # Batch size for training
LR = 0.1             # Learning rate for the optimizer
MAX_GRAD_NORM = 1.0  # Maximum L2 norm for gradient clipping (for DP)
DELTA = 1e-5         # Target delta value for ( $\epsilon$ ,  $\delta$ )-DP
LOG_DIR = "runs/dp_mnist" # TensorBoard log folder
MODEL_PATH = "SimpleNN_model.pth" # File to save the trained model

# -----
# Setup
# -----
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
os.makedirs(LOG_DIR, exist_ok=True)
writer = SummaryWriter(LOG_DIR)

# -----
# Dataset
# -----
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)) # Normalize MNIST grayscale
images
])
train_data = datasets.MNIST(root="./data", train=True, download=True,
transform=transform)
test_data = datasets.MNIST(root="./data", train=False, download=True,
transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=256, shuffle=False)

# -----
# Model - Simple Fully Connected Network
# -----
class SimpleNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28 * 28, 256)
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
```

```

        x = x.view(-1, 28 * 28) # Flatten input image
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

# -----
# Evaluation Function
# -----
def evaluate(model):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in test_loader:
            x, y = x.to(device), y.to(device)
            outputs = model(x)
            preds = outputs.argmax(dim=1)
            correct += (preds == y).sum().item()
            total += y.size(0)
    return correct / total

# -----
# Training Function
# -----
def train():
    model = SimpleNN().to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=LR)
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=2,
gamma=0.5)
    criterion = nn.CrossEntropyLoss()

    # Add Differential Privacy
    privacy_engine = PrivacyEngine()
    model, optimizer, train_loader_private = privacy_engine.make_private(
        module=model,
        optimizer=optimizer,
        data_loader=train_loader,
        noise_multiplier=1.1,
        max_grad_norm=MAX_GRAD_NORM,
    )

    print("Training with DP-SGD...")
    for epoch in range(1, EPOCHS + 1):
        model.train()
        total_loss = 0
        batch_count = 0
        for x, y in train_loader_private:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            outputs = model(x)
            loss = criterion(outputs, y)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            batch_count += 1

```

```

        avg_loss = total_loss / batch_count
        acc = evaluate(model)
        epsilon = privacy_engine.get_epsilon(DELTA)
        print(f"Epoch {epoch}: Avg Loss={avg_loss:.4f}, Accuracy={acc:.4f},
Epsilon={epsilon:.2f}")

        writer.add_scalar("Loss/train", avg_loss, epoch)
        writer.add_scalar("Accuracy/test", acc, epoch)
        writer.add_scalar("Privacy/epsilon", epsilon, epoch)

        scheduler.step()

    # Save model and optimizer
    torch.save({
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
    }, MODEL_PATH)
    print(f"Model saved to {MODEL_PATH}")
    writer.close()
    print("Training complete. Use TensorBoard to visualize logs:")
    print(f"tensorboard --logdir={LOG_DIR}")

# -----
# Testing Function
# -----
def test():
    model = SimpleNN().to(device)
    checkpoint = torch.load(MODEL_PATH, weights_only=False)
    state_dict = checkpoint['model_state_dict']
    new_state_dict = {k.replace('_module.', ''): v for k, v in
state_dict.items()}
    model.load_state_dict(new_state_dict)
    acc = evaluate(model)
    print(f"Loaded model test accuracy: {acc:.4f}")

# -----
# Call Functions
# -----
if __name__ == "__main__":
    # train()      # Uncomment to train the model
    test()        # Uncomment to test the saved model

```

CNN Script

```

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torch.utils.tensorboard import SummaryWriter
from opacus import PrivacyEngine

```

```

from opacus.validators import ModuleValidator
import os

# -----
# Hyperparameters
# -----
EPOCHS = 10
BATCH_SIZE = 64
LR = 0.1
MAX_GRAD_NORM = 1.0
DELTA = 1e-5
LOG_DIR = "runs/dp_mnist"
MODEL_PATH = "CNN_10Epoch.pth"

# -----
# Setup
# -----
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
os.makedirs(LOG_DIR, exist_ok=True)
writer = SummaryWriter(LOG_DIR)

# -----
# Dataset
# -----
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train_data = datasets.MNIST(root="./data", train=True, download=True,
transform=transform)
test_data = datasets.MNIST(root="./data", train=False, download=True,
transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=256, shuffle=False)

# -----
# CNN Model Architecture
# -----
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2) # Halves the spatial resolution
        self.dropout = nn.Dropout(0.25)
        self.fc1 = nn.Linear(64 * 14 * 14, 128) # After conv + pool, image is
14x14
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.dropout(x)
        x = x.reshape(x.size(0), -1) # Flatten
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)

```

```

        return x

# -----
# Evaluation Function
# -----
def evaluate(model):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in test_loader:
            x, y = x.to(device), y.to(device)
            outputs = model(x)
            preds = outputs.argmax(dim=1)
            correct += (preds == y).sum().item()
            total += y.size(0)
    return correct / total

# -----
# Training Function
# -----
def train():
    model = CNN()
    errors = ModuleValidator.validate(model)
    if errors:
        print("Model not valid for DP:", errors)
        model = ModuleValidator.fix(model)
        print("Model has been patched.")

    model = model.to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=LR)
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=2,
gamma=0.5)
    criterion = nn.CrossEntropyLoss()

    # Add differential privacy using Opacus
    privacy_engine = PrivacyEngine()
    model, optimizer, train_loader_private = privacy_engine.make_private(
        module=model,
        optimizer=optimizer,
        data_loader=train_loader,
        noise_multiplier=1.1,
        max_grad_norm=MAX_GRAD_NORM,
    )

    print("Training with DP-SGD (CNN)...")
    for epoch in range(1, EPOCHS + 1):
        model.train()
        total_loss = 0
        batch_count = 0
        for x, y in train_loader_private:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            outputs = model(x)

            # Explicitly handle potential batch size mismatches
            if outputs.size(0) != y.size(0):

```

```

        # If the sizes don't match, take only the first 'y.size(0)'
        elements from outputs
        outputs = outputs[:y.size(0)]

        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
        batch_count += 1

    avg_loss = total_loss / batch_count
    acc = evaluate(model)
    epsilon = privacy_engine.get_epsilon(DELTA)
    print(f"Epoch {epoch}: Avg Loss={avg_loss:.4f}, Accuracy={acc:.4f},
Epsilon={epsilon:.2f}")

    writer.add_scalar("Loss/train", avg_loss, epoch)
    writer.add_scalar("Accuracy/test", acc, epoch)
    writer.add_scalar("Privacy/epsilon", epsilon, epoch)

    scheduler.step()

    torch.save({
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
    }, MODEL_PATH)
    print(f"Model saved to {MODEL_PATH}")
    writer.close()
    print("Training complete. Use TensorBoard to visualize logs:")
    print(f"tensorboard --logdir={LOG_DIR}")

# -----
# Testing Function
# -----
def test():
    model = CNN().to(device)
    checkpoint = torch.load(MODEL_PATH, weights_only=False)
    state_dict = checkpoint['model_state_dict']
    new_state_dict = {k.replace('_module.', ''): v for k, v in
state_dict.items()}
    model.load_state_dict(new_state_dict)
    acc = evaluate(model)
    print(f"Loaded model test accuracy: {acc:.4f}")

# -----
# Call Functions Here
# -----
if __name__ == "__main__":
    # train() # Uncomment to train the model
    test() # Uncomment to test the saved model

```

No DP Script

[illegible]

```

transform=transform,
download=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# Initialize the model
model = SimpleNN().to(device)

# Define loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
print("Starting training...")
for epoch in range(10):
    model.train()
    running_loss = 0.0
    for i, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward pass and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        if (i + 1) % 100 == 0:
            print(f'Epoch [{epoch+1}/10], Step [{i+1}/{len(train_loader)}], Loss: {loss.item():.4f}')

# Evaluation
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f'\nTest Accuracy: {accuracy:.2f}%')

```